Модуль 3

Файловая система (fs)

Чтение • Запись • Потоки • Директории • path • Наблюдение • Практические задачи

3.1 Модуль fs — обзор

Модуль fs (file system) — один из самых используемых встроенных модулей Node.js. Он предоставляет функции для работы с файлами и директориями: чтение, запись, удаление, копирование, получение информации.

Модуль fs существует в трёх вариантах API:

- Синхронный (fs.readFileSync) — блокирует поток до завершения операции. Используйте только при старте приложения (чтение конфига)
- Callback (fs.readFile) — принимает функцию обратного вызова. Старый стиль, не рекомендуется для нового кода
- Promise (fs/promises) — возвращает Promise, работает с async/await. Рекомендуемый способ

В этом курсе мы будем использовать fs/promises как основной API, потому что он самый удобный и современный.

3.2 Чтение файлов

Рассмотрим все три способа чтения файла, чтобы вы понимали разницу:

```
import { readFileSync } from 'fs';
import { readFile } from 'fs/promises';


// 1. Синхронное чтение — блокирует Event Loop!
const data = readFileSync('config.json', 'utf-8');
console.log(JSON.parse(data));
```

Синхронное чтение останавливает выполнение всего кода, пока файл не будет прочитан. На сервере это означает, что все остальные запросы будут ждать. Используйте только при инициализации приложения.

```
// 2. Асинхронное чтение с Promise (рекомендуемый способ)
try {
    const content = await readFile('config.json', 'utf-8');
    const config = JSON.parse(content);
    console.log('Порт:', config.port);
} catch (err) {
    if (err.code === 'ENOENT') {
```

```
        console.error('Файл не найден');
    } else {
        console.error('Ошибка чтения:', err.message);
    }
}
```

Второй аргумент readFile — кодировка. Если указать "utf-8", функция вернёт строку. Без кодировки вернётся Buffer (бинарные данные) — это нужно для изображений, видео и других бинарных файлов.

Коды ошибок fs, которые полезно знать:

- ENOENT — файл или директория не существует (Error NO ENTry)
- EACCES — нет прав доступа
- EISDIR — попытка прочитать директорию как файл
- EEXIST — файл уже существует (при создании с флагом exclusive)

Пошаговый пример: синхронное и асинхронное чтение

Ниже — практический мини-эксперимент, который можно повторить у себя в проекте и увидеть разницу глазами.

Создайте папку для примера и перейдите в неё:

```
mkdir fs-read-demo
cd fs-read-demo
```

Создайте большой тестовый файл (чем больше файл, тем заметнее разница), просто запустите команду нижу целиком для создания огромного текстового файла для тестирования.

```
node -e "const fs=require('fs'); const chunk='A'.repeat(1024*1024); let
out=''; for(let i=0;i<120;i++) out+=chunk; fs.writeFileSync('big.txt',
out); console.log('big.txt created');"
```

Огромный файл big.txt создан, теперь попробуем прочитать его синхронно и асинхронно.

Инициализируем новый проект, запустив

```
npm init -y
```

Не забудьте добавить `"type": "module"` в package.json для включения ESM модулей

```
{
  "name": "fs-read-demo",
  "version": "1.0.0",
  "main": "sync-read.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "type": "module", // <-- этот параметр включает ESM модули
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

Теперь создайте файл sync-read.js с синхронным чтением:

```
import { readFileSync } from 'fs';

console.log('start sync');
setTimeout(() => console.log('timer fired (sync)'), 0);

const content = readFileSync('big.txt', 'utf-8');
console.log('sync length:', content.length);
console.log('end sync');
```

Запустите чтение синхронно:

```
node sync-read.js
```

Вывод будет таким:

```
start sync
sync length: 125829120
end sync
timer fired (sync)
```

Ожидаемая логика вывода:

- Сначала полностью выполнится readFileSync
- Только после этого сработает setTimeout -> 'timer fired (sync)'

- Это показывает, что Event Loop был заблокирован на время чтения

Теперь попробуем прочитать txt файл асинхронно. Для этого создайте новый файл async-read.js с асинхронным чтением:

```
import { readFile } from 'fs/promises';

console.log('start async');
setTimeout(() => console.log('timer fired (async)'), 0);

try {
    const content = await readFile('big.txt', 'utf-8');
    console.log('async length:', content.length);
    console.log('end async');
} catch (err) {
    console.error('read error:', err.message);
}
```

Запустите чтение асинхронно:

```
node async-read.js
```

Здесь вывод будет немного другим:

```
start async
timer fired (async)
async length: 125829120
end async
```

Ожидаемая логика вывода:

- 'timer fired (async)' может появиться раньше завершения чтения
- Поток Node.js не блокируется, пока операция чтения выполняется в фоне
- Приложение остаётся отзывчивым к другим задачам

Вывод по эксперименту:

- Синхронное чтение (readFileSync) проще по синтаксису, но блокирует Event Loop

- Асинхронное чтение (readFile / fs/promises) не блокирует Event Loop и подходит для серверов
- Для backend-кода по умолчанию всегда выбирайте асинхронный подход

Практический ориентир: readFileSync уместен в старте приложения (например, один раз прочитать config), а readFile — в обработчиках запросов и любых регулярных операциях с файлами.

3.3 Запись файлов

Функция writeFile создаёт файл или полностью перезаписывает его содержимое. Если файл не существует, он будет создан. Если существует — перезаписан без предупреждения.

```
import { writeFile, appendFile } from 'fs/promises';

// Записать текст (перезаписывает файл целиком)
await writeFile('output.txt', 'Привет, мир!\n', 'utf-8');

// Записать JSON с форматированием
const config = { port: 3000, host: 'localhost', debug: true };
await writeFile(
    'config.json',
    JSON.stringify(config, null, 2)  // null, 2 — отступ 2 пробела
);
```

JSON.stringify(obj, null, 2) — второй аргумент (null) это replacer-функция (обычно не нужна), третий (2) — количество пробелов для отступа. Без третьего аргумента JSON будет в одну строку.
Функция appendFile дописывает данные в конец файла, не удаляя существующее содержимое. Идеально для логов:

```
// Дописать в конец файла
await appendFile('log.txt', `[${new Date().toISOString()}] Сервер
запущен\n`);
```

Другие операции с файлами:

```
import { copyFile, rename, unlink } from 'fs/promises';

await copyFile('source.txt', 'copy.txt');   // Копирование
await rename('old.txt', 'new.txt');         // Переименование /
перемещение
await unlink('temp.txt');                   // Удаление файла
```

Функция rename() может не только переименовывать, но и перемещать файлы: rename("a/file.txt", "b/file.txt") переместит файл из директории a в b.

Пошаговый пример: запись, перезапись и дозапись файлов

Сделаем короткий практический сценарий, чтобы наглядно увидеть, чем отличаются writeFile и appendFile.

Создайте папку для эксперимента и перейдите в неё:

```
mkdir fs-write-demo
cd fs-write-demo
```

Инициализируйте проект как делали при чтении файлов, не забудьте про type: module

Создайте файл write-demo.js со следующим кодом:

```
import { writeFile, appendFile, readFile } from 'fs/promises';

console.log('1) Создаём notes.txt');
await writeFile('notes.txt', 'Первая версия текста\n', 'utf-8');
console.log(await readFile('notes.txt', 'utf-8'));

console.log('2) Перезаписываем notes.txt');
await writeFile('notes.txt', 'Вторая версия (старая удалена)\n', 'utf-
8');
console.log(await readFile('notes.txt', 'utf-8'));

console.log('3) Дописываем в конец файла');
await appendFile('notes.txt', 'Третья строка добавлена через
appendFile\n', 'utf-8');
console.log(await readFile('notes.txt', 'utf-8'));

console.log('4) Записываем JSON-отчёт');
```

```
const report = { savedAt: new Date().toISOString(), status: 'ok',
lines: 2 };
await writeFile('report.json', JSON.stringify(report, null, 2), 'utf-
8');

console.log('5) Пытаемся создать новый файл только если его ещё нет');
try {
    await writeFile('unique.txt', 'Создан один раз\n', { encoding:
'utf-8', flag: 'wx' });
    console.log('unique.txt создан');
} catch (err) {
    if (err.code === 'EEXIST') {
        console.log('unique.txt уже существует, файл не перезаписан');
    } else {
        console.error('Ошибка записи:', err.message);
    }
}
```

Запустите скрипт записи:

```
node write-demo.js
```

Проверьте содержимое файлов вручную, откройте файлы notes.txt и report.json и unique.txt

Что вы увидите в результате:

- После первого writeFile файл notes.txt создаётся
- Второй writeFile полностью заменяет старое содержимое
- appendFile добавляет строку в конец без удаления текущего текста
- Флаг wx защищает от случайной перезаписи уже существующего файла

Мини-сравнение sync и async записи:

Создайте sync-write.js:

```
import { writeFileSync } from 'fs';
console.log('start sync write');
setTimeout(() => console.log('timer fired (sync write)'), 0);
writeFileSync('sync.txt', 'SYNC DATA\n', 'utf-8');
console.log('end sync write');
```

Создайте async-write.js:

```
import { writeFile } from 'fs/promises';
console.log('start async write');
setTimeout(() => console.log('timer fired (async write)'), 0);
await writeFile('async.txt', 'ASYNC DATA\n', 'utf-8');
console.log('end async write');
```

Запустите оба варианта:

```
node sync-write.js
node async-write.js
```

Вывод sync-write

```
start sync write
end sync write
timer fired (sync write)
```

Вывод async-write

```
start async write
timer fired (async write)
end async write
```

Как видно порядок вывода немного разный, как и в синхронных и асинхронных операциях чтения.

Ожидаемая логика вывода:

- В sync-варианте setTimeout сработает после завершения writeFileSync
- В async-варианте setTimeout может сработать до сообщения 'end async write'
- Это показывает, что асинхронная запись не блокирует Event Loop

Практический ориентир: writeFile используйте для полной замены содержимого, appendFile — для логов и накопительных записей.

3.4 Работа с директориями

Для создания, чтения и удаления директорий используются функции mkdir, readdir и rm.

```
import { mkdir, readdir, rm, stat } from 'fs/promises';
import { existsSync } from 'fs';


// Создать директорию (recursive создаёт вложенные)
await mkdir('uploads', { recursive: true });
await mkdir('data/logs/2025', { recursive: true });

Опция recursive: true означает, что будут созданы все промежуточные
директории. Без неё, если директория data не существует,
mkdir("data/logs/2025") выбросит ошибку.

// Проверить существование файла или директории
if (existsSync('config.json')) {
    console.log('Файл существует');
}
```

existsSync — одна из немногих синхронных функций, которую допустимо использовать, потому что проверка существования файла — очень быстрая операция.

```
// Список файлов в директории
const files = await readdir('src');
console.log(files); // ['index.js', 'utils.js', ...]


// С информацией о типе (файл или директория)
const entries = await readdir('src', { withFileTypes: true });
for (const entry of entries) {
    const type = entry.isDirectory() ? 'DIR' : 'FILE';
    console.log(`${type}: ${entry.name}`);
}

Функция stat() возвращает подробную информацию о файле:

const info = await stat('package.json');
console.log('Размер:', info.size, 'байт');
console.log('Создан:', info.birthtime);
console.log('Изменён:', info.mtime);
console.log('Это директория:', info.isDirectory());
console.log('Это файл:', info.isFile());

// Удалить директорию со всем содержимым
await rm('temp', { recursive: true, force: true });
```

Флаг force: true подавляет ошибку, если директория не существует.

Пошаговый пример: создание, чтение и удаление директорий

Ниже — практический сценарий, где мы создаём структуру директорий, читаем содержимое, получаем метаданные и удаляем папки.

Создайте папку для примера и перейдите в неё:

```
mkdir fs-dir-demo
cd fs-dir-demo
```

Инициализируйте проект как мы делали раньше (опять же не забудьте про “type”: “module”)

Создайте файл dirs-demo.js со следующим кодом:

```
import { mkdir, readdir, rm, stat, writeFile } from 'fs/promises';


console.log('1) Создаём структуру директорий');


await mkdir('project/src/utils', { recursive: true });
await mkdir('project/logs/2026/03', { recursive: true });


console.log('2) Создаём тестовые файлы');


await writeFile('project/src/index.js', "console.log('hello from
index');\n", 'utf-8');
await writeFile('project/src/utils/math.js', 'export const
sum=(a,b)=>a+b;\n', 'utf-8');


console.log('3) Читаем содержимое project/src');


const files = await readdir('project/src');
console.log(files);
console.log('4) Читаем содержимое с типами');


const entries = await readdir('project/src', { withFileTypes: true });
for (const entry of entries) {
    const type = entry.isDirectory() ? 'DIR' : 'FILE';
    console.log(`${type}: ${entry.name}`);
}


console.log('5) Получаем метаданные файла index.js');
const info = await stat('project/src/index.js');
console.log('size:', info.size, 'bytes');
```

```
console.log('updated:', info.mtime.toISOString());
console.log('6) Удаляем временные логи');
await rm('project/logs', { recursive: true, force: true });
console.log('logs removed');
```

Запустите скрипт:

```
node dirs-demo.js
```

Проверьте результат в файловой системе. Структура директорий должна быть такая

```
.
├── dirs-demo.js
├── package.json
└── project
    └── src
        ├── index.js
        └── utils
            └── math.js
```

В конец файла dirs-demo.js добавьте проверку ошибки ENOENT (директория не существует):

```
try {
    await readdir('project/missing-folder');
} catch (err) {
    if (err.code === 'ENOENT') {
        console.log('Папка не найдена:', err.path);
    } else {
        throw err;
    }
}
```

Запустите node dirs-demo.js еще раз.

Вывод будет примерно такой

```
1) Создаём структуру директорий
2) Создаём тестовые файлы
3) Читаем содержимое project/src
[ '.DS_Store', 'index.js', 'utils' ]
4) Читаем содержимое с типами
FILE: .DS_Store
FILE: index.js
DIR: utils
5) Получаем метаданные файла index.js
size: 33 bytes
updated: 2026-03-04T06:00:24.021Z
6) Удаляем временные логи
logs removed
Папка не найдена: project/missing-folder
```

Вывод по эксперименту:

- mkdir с recursive: true создаёт всю вложенную структуру за один вызов
- readdir(dir) возвращает имена файлов и папок в директории
- readdir(dir, { withFileTypes: true }) даёт тип записи (файл/директория)
- stat() возвращает метаданные: размер, даты, тип файла
- rm(path, { recursive: true, force: true }) удаляет папку целиком и безопасно игнорирует отсутствие пути

Практический ориентир: для CLI-утилит и backend-задач используйте асинхронные функции fs/promises и явно обрабатывайте ошибки ENOENT/EACCES.

3.5 Потоки (Streams)

Представьте, что вам нужно обработать файл размером 2 ГБ. Функция readFile загрузит весь файл в оперативную память — это может привести к краху приложения. Потоки (Streams) решают эту проблему: они читают и обрабатывают данные порциями (chunks), обычно по 64 КБ.

В Node.js существует 4 типа потоков:

- Readable — поток чтения (чтение файла, получение HTTP-запроса)
- Writable — поток записи (запись в файл, отправка HTTP-ответа)
- Duplex — двунаправленный (TCP-сокет)
- Transform — преобразующий (сжатие, шифрование)

```
import { createReadStream, createWriteStream } from 'fs';
import { pipeline } from 'stream/promises';
import { createGzip } from 'zlib';

// Чтение файла потоком
const readStream = createReadStream('large-file.log', 'utf-8');

readStream.on('data', (chunk) => {
    console.log('Получена порция:', chunk.length, 'символов');
});

readStream.on('end', () => {
    console.log('Чтение завершено');
});

readStream.on('error', (err) => {
    console.error('Ошибка:', err.message);
});
```

Функция pipeline() — рекомендуемый способ соединения потоков. Она автоматически обрабатывает ошибки и закрывает потоки:

```
// Копирование большого файла потоком
const src = createReadStream('video.mp4');
const dest = createWriteStream('copy.mp4');
await pipeline(src, dest);
console.log('Файл скопирован');
```

Потоки можно соединять в цепочку. Например, чтение → сжатие → запись:

```
// Сжатие файла (gzip)
await pipeline(
    createReadStream('access.log'),    // Читаем
    createGzip(),                      // Сжимаем
    createWriteStream('access.log.gz') // Записываем
);
console.log('Файл сжат');
```

Преимущество потоков: файл любого размера обрабатывается с постоянным потреблением памяти (~64 КБ), а не загружается целиком.

Пошаговый пример: чтение, копирование и сжатие через потоки

Сделаем практику, где вы увидите потоки в действии: чтение по чанкам, копирование через pipeline и сжатие gzip.

Создайте рабочую папку и перейдите в неё:

```
mkdir fs-stream-demo
cd fs-stream-demo
```

Инициализируйте проект, запустив npm init -y и включив ESM модули

Сгенерируйте большой файл для эксперимента (примерно 120 МБ), запустив следующую команду:

```
node -e "const fs=require('fs'); const line='[INFO] stream test
line\n'; let out=''; for(let i=0;i<3_000_000;i++) out+=line;
fs.writeFileSync('big.log', out); console.log('big.log created');"
```

Создайте файл stream-read.js:

```
import { createReadStream } from 'fs';

let chunks = 0;
let bytes = 0;
const stream = createReadStream('big.log', { encoding: 'utf-8',
highWaterMark: 64 * 1024 });

stream.on('data', (chunk) => {
    chunks += 1;
    bytes += Buffer.byteLength(chunk, 'utf-8');
});

stream.on('end', () => {
    console.log('chunks:', chunks);
    console.log('bytes:', bytes);
});

stream.on('error', (err) => {
    console.error('read error:', err.message);
});
```

Запустите чтение потоком:

```
node stream-read.js
```

Вывод будет примерно такой:

```
chunks: 1099
bytes: 72000000
```

Теперь создайте файл stream-copy-gzip.js:

```
import { createReadStream, createWriteStream } from 'fs';
import { stat } from 'fs/promises';
import { pipeline } from 'stream/promises';
import { createGzip } from 'zlib';
console.time('copy');
await pipeline(
    createReadStream('big.log'),
    createWriteStream('big-copy.log')
);
console.timeEnd('copy');
console.time('gzip');
await pipeline(
    createReadStream('big.log'),
    createGzip(),
    createWriteStream('big.log.gz')
);
console.timeEnd('gzip');
const original = await stat('big.log');
const copy = await stat('big-copy.log');
const zipped = await stat('big.log.gz');
console.log('original:', original.size);
console.log('copy:', copy.size);
console.log('gzip:', zipped.size);
```

Запустите копирование и сжатие:

```
node stream-copy-gzip.js
```

Сравните размеры файлов

big-copy.log	Today at 12:41 PM	72 MB	Log File
big.log	Today at 12:38 PM	72 MB	Log File
big.log.gz	Today at 12:41 PM	175 KB	gzip co...archive

Сравните с readFile (увидите более высокий расход памяти):

Создайте readfile-compare.js:

```
import { readFile } from 'fs/promises';
const mb = (n) => (n / 1024 / 1024).toFixed(2) + ' MB';
console.log('heap before:', mb(process.memoryUsage().heapUsed));
const content = await readFile('big.log', 'utf-8');
console.log('length:', content.length);
console.log('heap after:', mb(process.memoryUsage().heapUsed));
```

Запустите:

```
node readfile-compare.js
```

Как интерпретировать расход памяти при readFile:

- Смотрите не на абсолютные значения, а на разницу между heap before и heap after.
- readFile загружает файл целиком, поэтому рост heapUsed обычно заметный и часто сопоставим с размером файла (иногда больше).
- Пиковое потребление памяти может быть выше финального значения heap after из-за промежуточных буферов.
- Если дельта кажется маленькой, это не всегда хорошо: мог отработать сборщик мусора (GC) или V8 переиспользовал уже выделенную память.
- Для более точной картины смотрите вместе три метрики: heapUsed (JS-объекты), external (Buffer), rss (вся память процесса).

Практический вывод: если на больших файлах память растёт сильно и скачкообразно, для продакшн-задач лучше переходить на createReadStream + pipeline.

Итоги эксперимента:

- Readable stream читает файл порциями, не загружая весь файл в память сразу
- pipeline корректно передаёт backpressure и безопасно закрывает потоки при ошибках

- big-copy.log будет того же размера, что и big.log, а big.log.gz обычно существенно меньше
- readFile на больших файлах чаще показывает более высокий расход heap-памяти

Практический ориентир: для больших файлов и сетевых данных используйте потоковую обработку и pipeline как базовый стандарт.

3.6 Наблюдение за файлами

Node.js позволяет отслеживать изменения файлов и директорий в реальном времени. Это используется в инструментах разработки (hot reload, live server):

```
import { watch } from 'fs/promises';

// Наблюдение за директорией (рекурсивно)
const watcher = watch('src', { recursive: true });
for await (const event of watcher) {
    console.log(`${event.eventType}: ${event.filename}`);
    // eventType: 'rename' (создание/удаление) или 'change' (изменение)
}
```

Именно так работает флаг --watch в Node.js 18+: он наблюдает за файлами и перезапускает скрипт при изменениях.

Пошаговый пример: логирование изменений в директории

Ниже — практический сценарий: поднимаем watcher, создаём изменения в файлах и интерпретируем события change/rename.

Подготовьте папку для эксперимента:

```
mkdir fs-watch-demo
cd fs-watch-demo
mkdir src
```

Инициализируйте проект используя npm init -y

Создайте watcher.js:

```
import { watch } from 'fs/promises';
const watcher = watch('src', { recursive: true });
const stopAfterMs = 120_000;
console.log('Watching ./src for 120 seconds...');


const timer = setTimeout(() => {
    watcher.return();
}, stopAfterMs);


try {
    for await (const event of watcher) {
        const file = event.filename ?? '<unknown>';
        const ts = new Date().toISOString();
        console.log(`[${ts}] ${event.eventType}: ${file}`);
    }
} finally {
    clearTimeout(timer);
    console.log('Watcher stopped');
}
```

Запустите watcher:

```
node watcher.js
```

Пока watcher работает в течение двух минут, в другом терминале создайте события:

```
echo "console.log('v1')" > src/app.js
echo "console.log('v2')" >> src/app.js
mv src/app.js src/main.js
rm src/main.js
mkdir src/utils
```

Как читать события:

- change — изменение содержимого существующего файла
- rename — создание, удаление или переименование записи
- одно действие может породить несколько событий подряд — это нормальное поведение файловой системы

Частый практический фильтр (логировать только .js):

```
for await (const event of watcher) {
    const file = event.filename ?? '';
    if (!file.endsWith('.js')) continue;
    console.log(`${event.eventType}: ${file}`);
}
```

Практические замечания:

- watch удобен для live reload, автосборки и логирования изменений в проекте
- поддержка recursive может зависеть от платформы; при необходимости наблюдайте поддиректории отдельно
- --watch в Node.js построен на этой же идее: отслеживание изменений + перезапуск процесса

Дополнительно: watcher можно создавать без ограничения по времени

Если нужно наблюдать директорию постоянно (например, в dev-режиме), уберите таймер и останавливайте процесс вручную через Ctrl+C.

Пример:

```
import { watch } from 'fs/promises';

const watcher = watch('src', { recursive: true });
console.log('Watching ./src... Press Ctrl+C to stop');

for await (const event of watcher) {
    const file = event.filename ?? '<unknown>';
    console.log(`${event.eventType}: ${file}`);
}
```

Практика

Задание 3.1:

- Напишите скрипт, который читает JSON-конфиг и выводит настройки в красивом формате
- Создайте простой логгер: функции log(msg), error(msg), warn(msg), которые дописывают записи в файл с датой и уровнем

- Напишите утилиту tree(dir), которая рекурсивно выводит дерево директорий с отступами
- Скопируйте большой файл двумя способами (readFile+writeFile и pipeline) — измерьте время и потребление памяти
- Создайте скрипт, который наблюдает за папкой и логирует все изменения

Задание 3.2 (вопросы для самопроверки):

- Какой API модуля fs является рекомендуемым для нового кода и почему?
- В каком случае допустимо использовать readFileSync в серверном приложении?
- Что вернёт readFile, если не указать кодировку, и зачем это может быть полезно?
- Чем отличаются ENOENT, EACCES и EISDIR? Приведите пример для каждой ошибки.
- В чём разница между writeFile и appendFile на практике?
- Как с помощью флага wx защититься от случайной перезаписи файла?
- Почему mkdir(path, { recursive: true }) удобнее при создании вложенных директорий?
- Что даёт опция withFileTypes: true в readdir и когда она особенно полезна?
- Какие данные можно получить через stat() и как это применить в реальном проекте?
- Для чего используется rm(path, { recursive: true, force: true }) и какие у этого подхода риски?
- Почему readFile плохо подходит для очень больших файлов?
- Какие преимущества даёт pipeline при работе с потоками по сравнению с ручным pipe?
- Что такое backpressure и почему он важен при копировании/обработке больших данных?

- Как интерпретировать heapUsed, external и rss при сравнении readFile и stream-подхода?
- Чем события change и rename отличаются при watch и почему rename может приходить чаще, чем ожидается?
- Почему одно файловое действие иногда порождает несколько событий watcher подряд?
- Как сделать watcher бесконечным и как корректно завершать его в dev-сценарии?
- В каких задачах модуля fs вы бы выбрали синхронный подход, а в каких только асинхронный? Обоснуйте.
- Составьте мини-утилиту: отслеживание папки uploads, логирование событий и ротация логов по дате.